

Logistic Regression with Scikit Learn (SGDClassifier)

This notebook explores Scikit Learn Logistic Regression using SGDClassifier. We train logistic regression models on a binary classification dataset while experimenting with different Shapes of dataset to do comparative analysis with FHE enabled Logistic Regression training

```
In [ ]: import pandas as pd

df_cleaned = pd.read_csv('/home/ccs-lab-f13/VanshFHE/SGDTrain/df_final_strategic.csv') # Replace with your a

print(df_cleaned.head())
df_cleaned.shape
```

```
In [2]: pip install seaborn xgboost
```

```
Defaulting to user installation because normal site-packages is not writeable
Requirement already satisfied: seaborn in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (0.13.2)
Requirement already satisfied: xgboost in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (1.6.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from seaborn) (1.26.4)
Requirement already satisfied: pandas>=1.2 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from seaborn) (2.0.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from seaborn) (3.10.0)
Requirement already satisfied: scipy in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from xgboost) (1.13.1)
Requirement already satisfied: contourpy>=1.0.1 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.1)
Requirement already satisfied: cycler>=0.10 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.56.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.4.8)
Requirement already satisfied: packaging>=20.0 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (24.1)
Requirement already satisfied: pillow>=8 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (11.1.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/lib/python3/dist-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.4.7)
Requirement already satisfied: python-dateutil>=2.7 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from pandas>=1.2->seaborn) (2024.2)
Requirement already satisfied: tzdata>=2022.1 in /home/ccs-lab-f13/.local/lib/python3.10/site-packages (from pandas>=1.2->seaborn) (2024.1)
Requirement already satisfied: six>=1.5 in /usr/lib/python3/dist-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.16.0)
Note: you may need to restart the kernel to use updated packages.
```

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

X = df_cleaned.drop('TARGET', axis=1)
y = df_cleaned['TARGET']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Training Logistic Regression for feature importance...")
```

```

lr = LogisticRegression(max_iter=1000, random_state=42)
lr.fit(X_train_scaled, y_train)

coefficients = np.abs(lr.coef_[0])

feature_importance = pd.DataFrame({
    'feature': X.columns,
    'importance': coefficients
}).sort_values('importance', ascending=False)

print("Training completed!")

```

Features shape: (307511, 363)
 Target shape: (307511,)
 Training Logistic Regression for feature importance...
 Training completed!

We train the logistic regression model for different samples, and for each sample space we observe model performance for various feature sets

Shape (100, 40/60/80)

```

In [6]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 50
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features =====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,

```

```

        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[:, 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n=== Summary Across All Feature Counts ===")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (100, 40)
Class distribution:
  TARGET
0      50
1      50
Name: count, dtype: int64
AUC: 0.6400 | Accuracy: 0.7500
Confusion Matrix:
[[8 2]
 [3 7]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.73	0.80	0.76	10
1	0.78	0.70	0.74	10
accuracy			0.75	20
macro avg	0.75	0.75	0.75	20
weighted avg	0.75	0.75	0.75	20

```

==== Using Top 60 Features ====
Balanced dataset shape: (100, 60)
Class distribution:
  TARGET
1      50
0      50
Name: count, dtype: int64
AUC: 0.7300 | Accuracy: 0.6000
Confusion Matrix:
[[6 4]
 [4 6]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.60	0.60	0.60	10
1	0.60	0.60	0.60	10
accuracy			0.60	20
macro avg	0.60	0.60	0.60	20
weighted avg	0.60	0.60	0.60	20

```

==== Using Top 80 Features ====
Balanced dataset shape: (100, 80)
Class distribution:
  TARGET
1      50
0      50
Name: count, dtype: int64
AUC: 0.8600 | Accuracy: 0.7000
Confusion Matrix:
[[9 1]
 [5 5]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.64	0.90	0.75	10
1	0.83	0.50	0.62	10
accuracy			0.70	20
macro avg	0.74	0.70	0.69	20
weighted avg	0.74	0.70	0.69	20

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.6400 | Accuracy: 0.7500
Features: 60 | AUC: 0.7300 | Accuracy: 0.6000
Features: 80 | AUC: 0.8600 | Accuracy: 0.7000

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (250, 40/60/80)

```
In [7]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 125
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features ====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[: , 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (250, 40)
Class distribution:
  TARGET
0      125
1      125
Name: count, dtype: int64
AUC: 0.7184 | Accuracy: 0.6200
Confusion Matrix:
[[10 15]
 [ 4 21]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.71	0.40	0.51	25
1	0.58	0.84	0.69	25
accuracy			0.62	50
macro avg	0.65	0.62	0.60	50
weighted avg	0.65	0.62	0.60	50

```

==== Using Top 60 Features ====
Balanced dataset shape: (250, 60)
Class distribution:
  TARGET
1      125
0      125
Name: count, dtype: int64
AUC: 0.6560 | Accuracy: 0.6200
Confusion Matrix:
[[15 10]
 [ 9 16]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.62	0.60	0.61	25
1	0.62	0.64	0.63	25
accuracy			0.62	50
macro avg	0.62	0.62	0.62	50
weighted avg	0.62	0.62	0.62	50

```

==== Using Top 80 Features ====
Balanced dataset shape: (250, 80)
Class distribution:
  TARGET
1      125
0      125
Name: count, dtype: int64
AUC: 0.6720 | Accuracy: 0.6000
Confusion Matrix:
[[14 11]
 [ 9 16]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.61	0.56	0.58	25
1	0.59	0.64	0.62	25
accuracy			0.60	50
macro avg	0.60	0.60	0.60	50
weighted avg	0.60	0.60	0.60	50

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.7184 | Accuracy: 0.6200
Features: 60 | AUC: 0.6560 | Accuracy: 0.6200
Features: 80 | AUC: 0.6720 | Accuracy: 0.6000

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (500, 40/60/80)

```
In [8]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 250
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features =====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[:, 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (500, 40)
Class distribution:
  TARGET
1    250
0    250
Name: count, dtype: int64
AUC: 0.7172 | Accuracy: 0.6300
Confusion Matrix:
[[25 25]
 [12 38]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.68	0.50	0.57	50
1	0.60	0.76	0.67	50
accuracy			0.63	100
macro avg	0.64	0.63	0.62	100
weighted avg	0.64	0.63	0.62	100

```

==== Using Top 60 Features ====
Balanced dataset shape: (500, 60)
Class distribution:
  TARGET
0    250
1    250
Name: count, dtype: int64
AUC: 0.6456 | Accuracy: 0.6300
Confusion Matrix:
[[31 19]
 [18 32]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.63	0.62	0.63	50
1	0.63	0.64	0.63	50
accuracy			0.63	100
macro avg	0.63	0.63	0.63	100
weighted avg	0.63	0.63	0.63	100

```

==== Using Top 80 Features ====
Balanced dataset shape: (500, 80)
Class distribution:
  TARGET
0    250
1    250
Name: count, dtype: int64
AUC: 0.6980 | Accuracy: 0.6700
Confusion Matrix:
[[34 16]
 [17 33]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.67	0.68	0.67	50
1	0.67	0.66	0.67	50
accuracy			0.67	100
macro avg	0.67	0.67	0.67	100
weighted avg	0.67	0.67	0.67	100

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.7172 | Accuracy: 0.6300
Features: 60 | AUC: 0.6456 | Accuracy: 0.6300
Features: 80 | AUC: 0.6980 | Accuracy: 0.6700

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (750, 40/60/80)

```
In [9]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 375
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features ====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[: , 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (750, 40)
Class distribution:
  TARGET
0      375
1      375
Name: count, dtype: int64
AUC: 0.7740 | Accuracy: 0.7267
Confusion Matrix:
[[54 21]
 [20 55]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.73	0.72	0.72	75
1	0.72	0.73	0.73	75
accuracy			0.73	150
macro avg	0.73	0.73	0.73	150
weighted avg	0.73	0.73	0.73	150

```

==== Using Top 60 Features ====
Balanced dataset shape: (750, 60)
Class distribution:
  TARGET
0      375
1      375
Name: count, dtype: int64
AUC: 0.8085 | Accuracy: 0.7400
Confusion Matrix:
[[58 17]
 [22 53]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.72	0.77	0.75	75
1	0.76	0.71	0.73	75
accuracy			0.74	150
macro avg	0.74	0.74	0.74	150
weighted avg	0.74	0.74	0.74	150

```

==== Using Top 80 Features ====
Balanced dataset shape: (750, 80)
Class distribution:
  TARGET
0      375
1      375
Name: count, dtype: int64
AUC: 0.6507 | Accuracy: 0.6133
Confusion Matrix:
[[44 31]
 [27 48]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.62	0.59	0.60	75
1	0.61	0.64	0.62	75
accuracy			0.61	150
macro avg	0.61	0.61	0.61	150
weighted avg	0.61	0.61	0.61	150

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.7740 | Accuracy: 0.7267
Features: 60 | AUC: 0.8085 | Accuracy: 0.7400
Features: 80 | AUC: 0.6507 | Accuracy: 0.6133

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (1000, 40/60/80)

```
In [10]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 500
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features ====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[:, 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (1000, 40)
Class distribution:
  TARGET
1    500
0    500
Name: count, dtype: int64
AUC: 0.7394 | Accuracy: 0.7000
Confusion Matrix:
[[71 29]
 [31 69]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.70	0.71	0.70	100
1	0.70	0.69	0.70	100
accuracy			0.70	200
macro avg	0.70	0.70	0.70	200
weighted avg	0.70	0.70	0.70	200

```

==== Using Top 60 Features ====
Balanced dataset shape: (1000, 60)
Class distribution:
  TARGET
1    500
0    500
Name: count, dtype: int64
AUC: 0.7129 | Accuracy: 0.6400
Confusion Matrix:
[[63 37]
 [35 65]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.64	0.63	0.64	100
1	0.64	0.65	0.64	100
accuracy			0.64	200
macro avg	0.64	0.64	0.64	200
weighted avg	0.64	0.64	0.64	200

```

==== Using Top 80 Features ====
Balanced dataset shape: (1000, 80)
Class distribution:
  TARGET
1    500
0    500
Name: count, dtype: int64
AUC: 0.6959 | Accuracy: 0.6250
Confusion Matrix:
[[69 31]
 [44 56]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.61	0.69	0.65	100
1	0.64	0.56	0.60	100
accuracy			0.62	200
macro avg	0.63	0.62	0.62	200
weighted avg	0.63	0.62	0.62	200

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.7394 | Accuracy: 0.7000
Features: 60 | AUC: 0.7129 | Accuracy: 0.6400
Features: 80 | AUC: 0.6959 | Accuracy: 0.6250

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (1500, 40/60/80)

```
In [11]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 750
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features =====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[:, 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (1500, 40)
Class distribution:
  TARGET
0      750
1      750
Name: count, dtype: int64
AUC: 0.6952 | Accuracy: 0.6367
Confusion Matrix:
[[100  50]
 [ 59  91]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.63	0.67	0.65	150
1	0.65	0.61	0.63	150
accuracy			0.64	300
macro avg	0.64	0.64	0.64	300
weighted avg	0.64	0.64	0.64	300

```

==== Using Top 60 Features ====
Balanced dataset shape: (1500, 60)
Class distribution:
  TARGET
0      750
1      750
Name: count, dtype: int64
AUC: 0.7301 | Accuracy: 0.6433
Confusion Matrix:
[[ 86  64]
 [ 43 107]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.67	0.57	0.62	150
1	0.63	0.71	0.67	150
accuracy			0.64	300
macro avg	0.65	0.64	0.64	300
weighted avg	0.65	0.64	0.64	300

```

==== Using Top 80 Features ====
Balanced dataset shape: (1500, 80)
Class distribution:
  TARGET
0      750
1      750
Name: count, dtype: int64
AUC: 0.7293 | Accuracy: 0.6667
Confusion Matrix:
[[ 94  56]
 [ 44 106]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.68	0.63	0.65	150
1	0.65	0.71	0.68	150
accuracy			0.67	300
macro avg	0.67	0.67	0.67	300
weighted avg	0.67	0.67	0.67	300

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.6952 | Accuracy: 0.6367
Features: 60 | AUC: 0.7301 | Accuracy: 0.6433
Features: 80 | AUC: 0.7293 | Accuracy: 0.6667

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
```

Shape (2000, 40/60/80)

```
In [12]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score, accuracy_score

feature_counts = [40, 60, 80]
n_samples_per_class = 1000
random_state = 42
results = []

np.random.seed(random_state)

for n_features in feature_counts:
    print(f"\n==== Using Top {n_features} Features =====")

    top_features = feature_importance.head(n_features)['feature'].tolist()
    X_selected = df_cleaned[top_features]
    y = df_cleaned['TARGET']

    target_1_indices = y[y == 1].index
    target_0_indices = y[y == 0].index

    target_1_sampled = np.random.choice(target_1_indices, size=n_samples_per_class, replace=False)
    target_0_sampled = np.random.choice(target_0_indices, size=n_samples_per_class, replace=False)

    balanced_indices = np.concatenate([target_1_sampled, target_0_sampled])
    np.random.shuffle(balanced_indices)

    X_balanced = X_selected.loc[balanced_indices]
    y_balanced = y.loc[balanced_indices]

    print(f"Balanced dataset shape: {X_balanced.shape}")
    print("Class distribution:\n", y_balanced.value_counts())

    X_train, X_test, y_train, y_test = train_test_split(
        X_balanced, y_balanced, test_size=0.2, random_state=random_state, stratify=y_balanced
    )

    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    sgd_model = SGDClassifier(
        loss="log_loss",
        penalty="l2",
        alpha=0.01,
        fit_intercept=True,
        max_iter=10,
        tol=0.01,
        shuffle=True,
        verbose=0,
        random_state=random_state,
        learning_rate="adaptive",
        eta0=0.001,
        power_t=0.5,
        early_stopping=True,
        validation_fraction=0.1,
        n_iter_no_change=2,
        class_weight="balanced",
        warm_start=False,
```

```

        average=False
    )

    sgd_model.fit(X_train_scaled, y_train)

    sgd_pred = sgd_model.predict(X_test_scaled)
    sgd_pred_proba = sgd_model.predict_proba(X_test_scaled)[: , 1]

    auc_score = roc_auc_score(y_test, sgd_pred_proba)
    acc_score = accuracy_score(y_test, sgd_pred)
    conf_matrix = confusion_matrix(y_test, sgd_pred)
    class_report = classification_report(y_test, sgd_pred, output_dict=True)

    print(f"AUC: {auc_score:.4f} | Accuracy: {acc_score:.4f}")
    print("Confusion Matrix:")
    print(conf_matrix)
    print("Classification Report:")
    print(classification_report(y_test, sgd_pred))

    results.append({
        "n_features": n_features,
        "auc": auc_score,
        "accuracy": acc_score,
        "confusion_matrix": conf_matrix,
        "classification_report": class_report
    })

print("\n==== Summary Across All Feature Counts ====")
for r in results:
    print(f"Features: {r['n_features']} | AUC: {r['auc']:.4f} | Accuracy: {r['accuracy']:.4f}")

```

```

==== Using Top 40 Features ====
Balanced dataset shape: (2000, 40)
Class distribution:
  TARGET
1    1000
0    1000
Name: count, dtype: int64
AUC: 0.7089 | Accuracy: 0.6625
Confusion Matrix:
[[135  65]
 [ 70 130]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.66	0.68	0.67	200
1	0.67	0.65	0.66	200
accuracy			0.66	400
macro avg	0.66	0.66	0.66	400
weighted avg	0.66	0.66	0.66	400

```

==== Using Top 60 Features ====
Balanced dataset shape: (2000, 60)
Class distribution:
  TARGET
1    1000
0    1000
Name: count, dtype: int64
AUC: 0.6891 | Accuracy: 0.6325
Confusion Matrix:
[[118  82]
 [ 65 135]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.64	0.59	0.62	200
1	0.62	0.68	0.65	200
accuracy			0.63	400
macro avg	0.63	0.63	0.63	400
weighted avg	0.63	0.63	0.63	400

```

==== Using Top 80 Features ====
Balanced dataset shape: (2000, 80)
Class distribution:
  TARGET
1    1000
0    1000
Name: count, dtype: int64
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: Conver
genceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve
the fit.
  warnings.warn(
AUC: 0.7080 | Accuracy: 0.6425
Confusion Matrix:
[[122  78]
 [ 65 135]]
Classification Report:

```

	precision	recall	f1-score	support
0	0.65	0.61	0.63	200
1	0.63	0.68	0.65	200
accuracy			0.64	400
macro avg	0.64	0.64	0.64	400
weighted avg	0.64	0.64	0.64	400

```

==== Summary Across All Feature Counts ====
Features: 40 | AUC: 0.7089 | Accuracy: 0.6625
Features: 60 | AUC: 0.6891 | Accuracy: 0.6325
Features: 80 | AUC: 0.7080 | Accuracy: 0.6425

```

```
/home/ccs-lab-f13/.local/lib/python3.10/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: ConvergenceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve the fit.  
  warnings.warn(
```

We compared the observations of Scikit Learn SGDClassifier with the Concrete ML FHE enabled Classifier

In []: